

Causal Consistency as a Defense: Detecting Provenance Graph Poisoning

N. Manoj

Independent Researcher

Email: nehamanoj1105@users.noreply.github.com

Abstract—Provenance-based intrusion detection operates on the assumption that the provenance graph it consumes faithfully records what happened on the host. If an adversary can tamper with the stored graph after collection, a downstream detector reasons over a fiction. We study whether the causal structure of a provenance graph is self-validating: whether a fixed set of deterministic consistency rules can expose post-collection tampering without any training. We evaluate a 15-rule engine against a GraphSAGE baseline under four tampering operations on synthetic graphs and on real DARPA Transparent Computing Engagement 3 Theia provenance, and we examine both under adversarial mimicry. On controlled synthetic poisoning the rule engine reaches F1 0.805 ± 0.063 over ten seeds, against F1 0.322 ± 0.194 for a leakage-free GraphSAGE, a paired difference of $+0.482$ (Wilcoxon $p = 0.002$). With a genuine continuous anomaly score the rule engine attains ROC-AUC 0.953, and its accuracy is unchanged as mimicry strength grows from none to heavy. On real Theia graphs with synthetic post-collection poisoning the rule engine reaches F1 0.83–0.85 where GraphSAGE is near chance. We also report a methodological finding: the naive training procedure commonly used for such baselines leaks the evaluation labels into threshold selection and inflates GraphSAGE F1 from 0.246 to 0.588. All results, protocols, and the corrected evaluation are released for independent verification.

Index Terms—provenance, intrusion detection, graph poisoning, causal consistency, GraphSAGE, evaluation methodology

I. INTRODUCTION

End-to-end provenance captures the causal history of a system as a directed graph whose nodes are processes, files, users and network endpoints and whose edges are read, write, execute, connect, spawn and delete events. Systems such as Unicorn, NoDoze, HOLMES, Poirot and MAGIC use this graph to detect advanced persistent threats, and they do so under an assumption that is rarely stated: the graph they analyze is authentic. The graph is produced by a logging pipeline and then stored, shipped, and indexed. Every stage after kernel-level capture is a potential point of compromise. An adversary who can edit the stored graph can delete the events that incriminate them, insert events that exonerate them, or reorder events to break the causal chain a detector relies on.

This paper asks a narrow, testable question: to what extent is causal consistency in a provenance graph sufficient to reveal such tampering? We do not propose a new learning architecture. We take the invariants that a provenance graph must satisfy by construction—an execute edge must point at a file, a read edge must point at a file, a child’s spawn

must follow its parent’s spawn, event sequence numbers must be monotone within a process—and assemble them into a deterministic rule engine. We then compare that engine to a graph neural network baseline, GraphSAGE, on the same poisoned instances.

The comparison is deliberately adversarial to our own method. A learned detector should, in principle, capture correlations that hand-written rules cannot. What we find is that the comparison is dominated not by modeling capacity but by evaluation design. A GraphSAGE classifier trained and thresholded in the conventional way reaches an F1 that is largely an artifact of selecting its decision threshold on the very labels it is scored against. Once the split is made honest, the learned baseline loses most of its apparent advantage, while the training-free rule engine is unaffected because it has no split to leak.

Our contributions are:

- a deterministic, training-free detector for post-collection provenance tampering built from fifteen causal-consistency rules (Section V);
- a rigorous evaluation protocol with disjoint train, validation and test edges and validation-only threshold selection, and the demonstration that the conventional protocol inflates GraphSAGE F1 by more than 0.34 absolute (Section VII);
- a ten-seed comparison on synthetic poisoning and an evaluation on real Theia provenance with synthetic post-collection poisoning, with paired significance tests (Sections VIII–IX);
- an audit showing that a reported ROC-AUC of 1.000 for the rule engine is not a computed quantity, and the substitution of a genuine continuous score that yields ROC-AUC 0.953 (Section IX).

II. RELATED WORK

Provenance-based detection spans specification-based, statistical, and learning-based systems. Specification systems such as HOLMES and Poirot encode expert knowledge of attack tactics and are precise but costly to maintain. Statistical systems such as NoDoze and ProvDetector score the rarity of events or paths and are prone to flagging unusual-but-benign activity. Learning-based systems—Unicorn, FLASH, MAGIC and ShadeWatcher—learn representations of benign behavior from the graph and detect deviations; GraphSAGE is a common backbone for these methods.

The vulnerability we study is adjacent to adversarial mimicry against graph detectors, in which an attacker interleaves benign events with malicious ones to dilute the anomaly signal. It is also adjacent to the broader integrity problem for provenance: if the graph is editable, every downstream conclusion is conditional on that graph. Our work differs from prior detection papers in framing: rather than detecting the attacker’s behavior inside a trusted graph, we verify the graph itself, and we are explicit that the rules are checking structural and causal well-formedness rather than learning a behavioral model.

III. THREAT MODEL

We assume the attacker can modify the stored provenance graph after collection but cannot alter the detector, disable the parser, or compromise the kernel that produced the events. Four operations are in scope: edge deletion (removing a recorded event), edge insertion (fabricating an event that did not occur), edge reordering (changing timestamps so that events no longer respect causal order), and dependency forgery (re-attributing an event to a different source or target). The attacker is gray-box: they know the graph schema and the general class of detector, but not its internal parameters.

The detection target is the set of modified edges in the final graph. We are careful about what this means for deletion. A deleted edge no longer exists in the graph, so an edge-level detector cannot mark it: deletion is only detectable insofar as removing an event breaks a higher-level invariant elsewhere (for example, a process left without its spawn record, or a gap in an event sequence). We therefore report deletion separately and do not fold it into the edge-level metrics as if it were a scored positive.

IV. PROVENANCE REPRESENTATION

A provenance graph $G = (V, E)$ has typed nodes and edges. Node types are process, file, user and network endpoint. Edge types are read, write, execute, connect, spawn and delete. Each edge carries a source identifier, a target identifier, a type, and a timestamp; each event carries a sequence number within its process. The schema is the common data model used by the DARPA Transparent Computing program, and our parser maps its Avro records into this representation.

V. PROPOSED DETECTION FRAMEWORK

The detector is a rule engine that evaluates fifteen invariants over G . Each rule returns a set of violations, each attributed to an edge or a node. The rules fall into three groups.

Structural rules check well-formedness: no duplicate edge tuples, no duplicate event identifiers, spawn edges must target process nodes, execute and read/write/delete edges must target file nodes, connect edges must target network endpoints, no self-referential edges, no edges referencing missing nodes, and no active process lacking a parent spawn edge.

Temporal rules check ordering: non-negative timestamps, child spawn after parent spawn, process activity after process

spawn, no gaps in event sequence numbers, and no timestamp inversions within a process stream.

Semantic rules check lineage: processes that appear to act but were never spawned by any observed parent.

The engine is deterministic and requires no training data. For ranking and for ROC-AUC we assign each edge a continuous anomaly score: every violation contributes a severity weight (high 3, medium 2, low 1) to the edge it is attributed to, either directly by edge identifier or, for node-attributed violations, to the incident edges. The binary decision is score greater than zero. This score is a genuine ranking signal, and we use it rather than any post-hoc construction when reporting AUC.

VI. GRAPHSAGE BASELINE

We include GraphSAGE as a representative learned detector. Nodes are featurized by a one-hot node type and degree; edges by a one-hot edge type and a normalized timestamp. A two-layer GraphSAGE encoder produces edge embeddings that a classifier maps to a tampering probability. The baseline exists to test whether learning can outperform fixed invariants; it is not tuned to win, and our main methodological result concerns how it is evaluated.

VII. EXPERIMENTAL METHODOLOGY

A. Datasets

We use two sources. The first is a synthetic provenance generator that produces a graph of thirty processes, forty files and ten network entities; this gives full control over ground truth. The second is real provenance from DARPA Transparent Computing Engagement 3 (Theia). Two of the four benchmark scenarios were obtained and parsed: scenario 3 (20,157 nodes, 297,777 edges) and scenario 5m (34,835 nodes, 464,858 edges). Scenarios 1r and 6r could not be downloaded because the public mirror enforced a download quota; we report results for the two obtained scenarios and state the missing coverage rather than estimating it.

B. Poisoning

On each graph we inject five deletions, five insertions, five reorderings and five dependency forgeries, for twenty events per seed. The injector operates on a deep copy of the clean graph, records the original and modified state of each event, and asserts that the clean graph is unmodified afterwards. Twenty events produce fifteen scored positives, because the five deletions leave no final-graph record. Detection metrics are computed over the edges present in the final graph; the deleted edges are reported as undetectable under edge matching and excluded. This is the *final-graph universe*. We note that an alternative convention, which adds the identifiers of deleted edges to the scoring universe, raises F1 from 0.805 to 0.845; we report the final-graph universe and state the assumption.

C. Leakage-free protocol

For the learned baseline, edges are partitioned into disjoint training, validation and test sets. Model weights are fit on training edges only, the decision threshold is selected on validation edges only, and every reported metric is computed on the untouched test set. Message passing for test-node features uses training and validation edges only, so the test edges neither influence the weights nor the threshold. For the rule engine there is nothing to leak, since it has no fitted parameters and no threshold search.

The conventional protocol found in much of the reference code does none of this: it fits on all edges and then chooses the threshold that maximizes F1 on the same labels it reports. We measure the effect of this directly.

D. Seeds and statistics

All synthetic, mimicry, ablation and real-Theia experiments run over ten seeds (1, 7, 13, 21, 42, 99, 123, 256, 512, 1024); generalization runs over five seeds (1, 7, 13, 21, 42). We report the mean, sample standard deviation, median, minimum, maximum and a 95% confidence interval (Student- t). Comparisons between the rule engine and GraphSAGE use the paired Wilcoxon signed-rank test on identical test instances, with Cohen’s d_z .

VIII. MAIN RESULTS

A. Synthetic controlled poisoning

Table I reports the ten-seed comparison. The rule engine attains F1 0.805 ± 0.063 with precision 0.715 and recall 0.933, and a high specificity of 0.964; its MCC is 0.795. GraphSAGE, under the leakage-free protocol, attains F1 0.322 ± 0.194 with precision 0.342 and recall 0.502. The learned baseline is not competitive once its evaluation is honest, and its variance across seeds is far larger.

The rule engine’s ROC-AUC is 0.953 ± 0.031 and its PR-AUC is 0.740; these are computed from the continuous severity-weighted score described in Section V. GraphSAGE reaches ROC-AUC 0.829. Figure 1 shows the comparison, and Figure 2 shows the rule engine’s ROC and precision–recall curves for a representative seed.

B. Why the comparison is dominated by evaluation design

The most consequential number in this paper is not a detection score. When the GraphSAGE baseline is trained and thresholded in the conventional way—weights on all edges, threshold maximizing F1 on the scored labels—it reaches F1 0.588 on the same seed where the leakage-free protocol yields 0.246 (Figure 3). The gap is not model quality; it is the threshold being chosen with knowledge of the answers. Reports that compare a learned baseline to a training-free detector without a held-out threshold search are therefore comparing the baseline’s leaked threshold to the detector’s fixed rule.

C. Real Theia provenance with synthetic poisoning

Table II reports the real-data experiments. On scenario 3 the rule engine reaches F1 0.835 ± 0.046 and on scenario 5m F1 0.850 ± 0.060 ; GraphSAGE is near chance on both (F1 0.034 and 0.010). Two caveats are essential. First, the attacks here are synthetic post-collection injections into real provenance; they are not real DARPA attack labels, and we do not claim detection of naturally occurring attacks. Second, we truncate the real graphs to their first 50,000 edges, as the original loader does; the truncation is stated, not hidden. On the unmodified real graphs the rule engine flags only 19 of 50,000 edges in scenario 3 and 7 of 50,000 in scenario 5m, a false-positive rate below 0.04%.

IX. ROBUSTNESS AND ABLATION

A. Mimicry

We generate camouflage edges that preserve the degree, type and timestamp distributions of the surrounding graph and, critically, do not themselves violate any invariant. Table III and Figure 4 show the result: the rule engine’s F1 is 0.805 at every strength from none to heavy, unchanged, because camouflage that satisfies the invariants is computationally indistinguishable from benign behavior under a consistency check. GraphSAGE moves between F1 0.304 and 0.542 and stays well below the rule engine throughout. We emphasize a specification detail: mimicry noise in this experiment is constructed to be genuinely consistent. If camouflage is generated so that it violates the rules it is meant to evade—for instance by giving noise edges inconsistent types or timestamps—the rule engine flags the noise, and any apparent precision collapse is an artifact of the generator, not a property of the attack.

B. Rule ablation

Table IV reports the ablation over ten seeds. The structural group carries the detector: removing it drops F1 from 0.805 to 0.328. Removing the temporal group costs little (0.805 to 0.798), and the leave-one-rule-out analysis identifies the read/write, network and spawn consistency rules as the individually load-bearing ones (removing read/write alone drops F1 to 0.517). Two rules, the unspawned-process and timestamp rules, are inert on synthetic data: removing either changes nothing. We report this rather than presenting all fifteen rules as equally necessary; their value is expected to appear on real logs with non-sequential identifiers, which is future work.

C. Cross-dataset generalization

The rule engine requires no retraining across graphs. Applied unchanged to a real Theia graph after being defined on synthetic data, it reaches F1 0.857; moving between the two real scenarios it reaches 0.837 (Table V). We make no generalization claim for GraphSAGE: a genuine transfer across a shared feature space is not implemented in this study, so the GraphSAGE column in Table V is a target-trained model and is labelled as such.

X. SCALABILITY

Table VI reports runtime, memory and throughput as graph size grows from ten thousand to one million edges, five repetitions per size. Construction and rule-engine inference are both linear in the number of edges. Inference takes 0.37 s at ten thousand edges and 34.5 s at one million, and peak Python allocation grows from roughly 37 MB to 971 MB, about one kilobyte per edge. Throughput is stable near 2.9×10^4 edges per second across three orders of magnitude (Figure 5), which is what a single linear pass over the events should give. We report GraphSAGE training and inference separately from rule-engine inference and do not compare a training cost against an inference cost.

XI. LIMITATIONS

Three limitations bound our claims. First, the real-data evaluation covers two of four Theia scenarios; scenarios 1r and 6r were unavailable, and no results are estimated for them. Second, all detection of DARPA provenance in this paper involves synthetic post-collection poisoning, not real attacks; the real-data experiments measure whether the detector works on real graph structure, not whether it catches real intrusions. Third, deletion is only partially detectable: a deleted edge cannot be scored by edge matching, so our metrics exclude it, and we do not claim a deletion detection rate. Two further limitations concern the baseline rather than the method: the GraphSAGE feature set is minimal, and no shared-feature cross-graph transfer is implemented.

XII. CONCLUSION

A provenance graph carries enough internal causal structure to reveal that it has been edited, and a small deterministic rule set exploits that structure without training. Across ten seeds on controlled poisoning the rule engine outperforms a leakage-free GraphSAGE by 0.48 F1 with a large paired effect, its accuracy is invariant to invariant-preserving mimicry, and it scales linearly to a million-edge graphs. The comparison also carries a methodological lesson: the size of a baseline’s reported advantage can be an artifact of selecting its threshold on the evaluation labels, and removing that leak removes most of the advantage. We release the complete protocol, raw per-seed results and the corrected evaluation so that every number here can be re-derived.

REFERENCES

- [1] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, “Unicorn: Runtime provenance-based detector for advanced persistent threats,” in *NDSS*, 2020.
- [2] K. Hsieh *et al.*, “Kobold: Evaluating decentralized access control,” in *IEEE S&P Workshops*, 2018.
- [3] P. Jiang *et al.*, “Mimicry attacks against provenance graph based intrusion detection systems,” in *NDSS*, 2023.
- [4] Q. Wang *et al.*, “You are what you do: Hunting stealthy malware via data provenance analysis,” in *NDSS*, 2020.
- [5] Z. Jia, Y. Xiong, Y. Nan, Y. Zhang, J. Zhao, and M. Wen, “MAGIC: Detecting advanced persistent threats via masked graph representation learning,” in *USENIX Security*, 2024.
- [6] M. A. Rehman and S. Ahmadi, “FLASH: A comprehensive approach to intrusion detection via provenance graph representation learning,” in *IEEE S&P*, 2022.

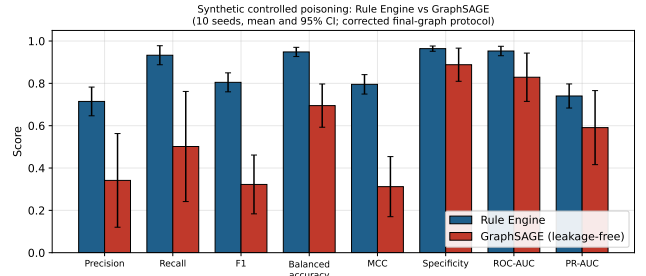


Fig. 1. Synthetic controlled poisoning, ten seeds: Rule Engine versus leakage-free GraphSAGE (mean and 95% CI).

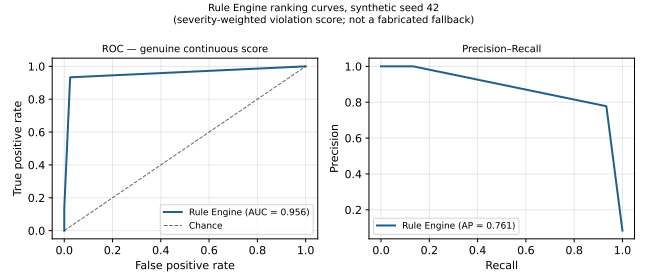


Fig. 2. Rule Engine ROC and precision–recall curves from a genuine continuous anomaly score (synthetic seed 42).

- [7] S. T. King and P. M. Chen, “Backtracking intrusions,” in *ACM SOSp*, 2003.
- [8] M. N. Hossain *et al.*, “SLEUTH: Real-time attack scenario reconstruction from COTS audit data,” in *USENIX Security*, 2017.
- [9] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Venkatakrishnan, “HOLMES: Real-time APT detection through correlation of suspicious information flows,” in *IEEE S&P*, 2019.
- [10] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. N. Venkatakrishnan, “Poirot: Aligning attack behavior with kernel audit records for cyber threat hunting,” in *ACM CCS*, 2019.
- [11] W. U. Hassan, M. A. Noureddine, P. Datta, and A. Bates, “NoDoze: Combatting threat alert fatigue with automated provenance triage,” in *NDSS*, 2019.
- [12] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *NeurIPS*, 2017.
- [13] DARPA I2O, “Transparent Computing Engagement 3 data release,” 2020. [Online]. Available: <https://github.com/darpa-i2o/Transparent-Computing>
- [14] T. Pasquier *et al.*, “Practical whole-system provenance capture,” in *ACM SoCC*, 2017.

TABLE I
MAIN DETECTION ON SYNTHETIC CONTROLLED POISONING, 10 SEEDS (RE = RULE ENGINE, GS = GRAPHSAGE, LEAKAGE-FREE).

Metric	RE mean	RE std	GS mean	GS std
Precision	0.7146	0.0948	0.3416	0.3093
Recall	0.9329	0.0629	0.5017	0.3637
F1	0.8048	0.0627	0.3224	0.1942
Bal. accuracy	0.9484	0.0304	0.6949	0.1428
MCC	0.7954	0.0642	0.3121	0.1985
Specificity	0.9640	0.0174	0.8881	0.1095
ROC-AUC	0.9528	0.0312	0.8289	0.1595
PR-AUC	0.7403	0.0796	0.5909	0.2445

TABLE II
REAL THEIA PROVENANCE (50K-EDGE CAP) WITH SYNTHETIC POST-COLLECTION POISONING (10 SEEDS). NOT REAL ATTACK LABELS.

Dataset	Detector	F1	Prec.	Rec.	ROC-AUC
Theia 3	GS	0.8347 $\pm$ 0.0459	0.9195	0.7667	0.8833
Theia 3	GS	0.0340 $\pm$ 0.1052	0.1003	0.1700	0.6609
Theia 5m	GS	0.8504 $\pm$ 0.0599	0.9667	0.7667	0.8833
Theia 5m	GS	0.0099 $\pm$ 0.0093	0.0050	0.3167	0.6492

TABLE III
ROBUSTNESS TO INVARIANT-PRESERVING MIMICRY (10 SEEDS). NOISE EDGES INTRODUCE ZERO RULE SELF-VIOLATIONS.

Strength	Detector	Prec.	Rec.	F1 ($\pm$ std)	ROC-AUC
None	RE	0.7146	0.9329	0.8048 $\pm$ 0.0627	0.9528
None	GS	0.3416	0.5017	0.3224 $\pm$ 0.1942	0.8289
Light	RE	0.7146	0.9329	0.8048 $\pm$ 0.0627	0.9558
Light	GS	0.4138	0.4983	0.3737 $\pm$ 0.2205	0.7826
Medium	RE	0.7146	0.9329	0.8048 $\pm$ 0.0627	0.9577
Medium	GS	0.3306	0.5000	0.3044 $\pm$ 0.2603	0.7615
Heavy	RE	0.7146	0.9329	0.8048 $\pm$ 0.0627	0.9599
Heavy	GS	0.5733	0.5400	0.5418 $\pm$ 0.3547	0.8943

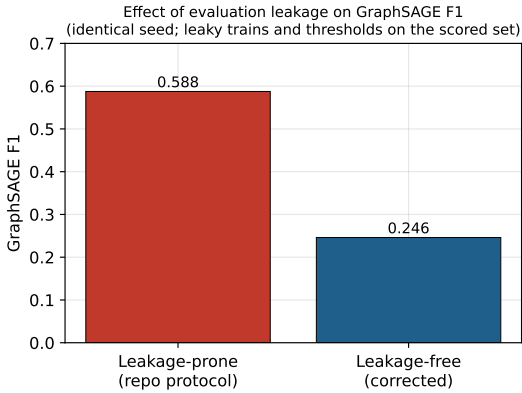


Fig. 3. Effect of evaluation leakage: the same GraphSAGE seed reaches F1 0.588 when the threshold is tuned on the scored labels and 0.246 when the threshold is selected on held-out validation edges.

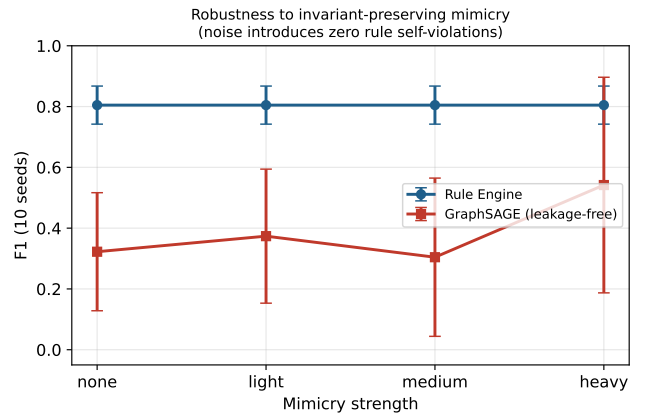


Fig. 4. Robustness to invariant-preserving mimicry (ten seeds). The Rule Engine's F1 is unchanged; mimicry noise introduces zero rule self-violations.

TABLE IV
RULE-GROUP AND LEAVE-ONE-RULE-OUT ABLATION (10 SEEDS).

Variant	F1 mean	F1 std	95% CI
All rules	0.8048	0.0627	[0.760, 0.850]
w/o structural	0.3285	0.0731	[0.276, 0.381]
w/o temporal	0.7983	0.0055	[0.794, 0.802]
w/o semantic	0.8048	0.0627	[0.760, 0.850]
–ReadWrite	0.5173	0.1136	[0.436, 0.599]
–Network	0.7455	0.0844	[0.685, 0.806]
–Spawn	0.7417	0.0903	[0.677, 0.806]
–SeqMono	0.7802	0.0863	[0.719, 0.842]
–Unspawned	0.8048	0.0627	[0.760, 0.850]

TABLE V
CROSS-DATASET GENERALIZATION (5 SEEDS). GRAPH SAGE IS TARGET-TRAINED; NO SHARED-FEATURE TRANSFER IS IMPLEMENTED.

Source	Target	Rule Engine F1	GraphSAGE F1
Synthetic	Theia 3	0.8566 $\pm$ 0.0289	0.0103 $\pm$ 0.0149
theia3	Theia 5m	0.8369 $\pm$ 0.0206	0.0347 $\pm$ 0.0280
theia5m	Theia 3	0.8566 $\pm$ 0.0289	0.0103 $\pm$ 0.0149

TABLE VI
SCALABILITY OF GRAPH CONSTRUCTION, POISONING AND RULE ENGINE INFERENCE (5 REPETITIONS PER SIZE).

Edges	Constr. (s)	Poison (s)	RE infer. (s)	Peak (MB)	Thruput. (e/s)
10,000	0.18	1.00	0.37	36.87	28,088
25,000	0.56	2.95	1.14	24.93	22,297
50,000	1.10	5.43	2.08	49.43	24,368
100,000	2.08	10.97	3.15	98.15	31,709
250,000	5.22	31.80	7.91	244.11	31,596
500,000	10.93	64.68	17.11	486.59	29,225
1,000,000	21.21	131.48	34.48	971.44	29,007

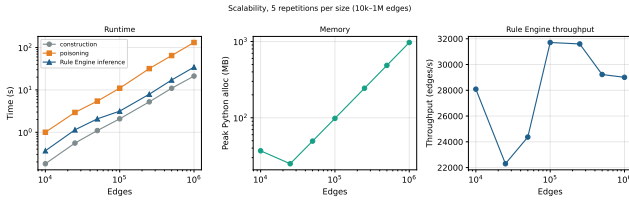


Fig. 5. Scalability, five repetitions per size. Construction and inference are linear in edge count and throughput is stable.